

接

三大功能模块技术文档

需求分析智能体 · 报价卡 · 需求 Demo 生成

「接单吧」OPC 撮合交易平台 · 技术介绍文档

版本：v1.0 | 编写日期：2026 年 8 月 13 日 | 面向读者：技术与产品团队、对外合作伙伴

目录

概述

第一章 需求分析智能体

- 1.1 功能定位与业务价值
- 1.2 整体流程
- 1.3 核心机制说明（ReAct 循环、工具集、accumulated 权威数据、自动收尾）
- 1.4 前后端交互方式（SSE 流式协议、form_suggestion_json 回填）
- 1.5 数据存储结构
- 1.6 权限与触发条件（场景白名单）

第二章 报价卡

- 2.1 功能定位与业务价值
- 2.2 整体流程
- 2.3 核心机制说明（品类→维度→档位配置、四层计价模型）
- 2.4 前后端交互方式（接口清单、展示与多卡对比）
- 2.5 数据存储结构
- 2.6 权限与通知机制

第三章 需求 Demo 生成

- 3.1 功能定位与业务价值
- 3.2 整体流程（三种触发方式）
- 3.3 核心机制说明（三段式 AI 流程）
- 3.4 前后端交互方式（接口清单、轮询预览）
- 3.5 数据存储结构与版本管理
- 3.6 权限与触发条件

概述

「接单吧」是一个 OPC（One Person Company，一人公司/独立专业服务者）撮合交易平台：发单方（客户）发布需求，平台运营撮合，OPC 承接执行。围绕「需求怎么写清楚、价格怎么算明白、方案长什么样」这三个交易前的核心痛点，平台构建了三大 AI/结构化功能模块：

模块	解决的问题	技术形态
需求分析智能体	客户说不清需求，需求文档不专业、信息缺漏	SSE 流式对话 + ReAct 工具调用循环的 LLM 智能体，最终一键回填发布表单
报价卡	报价不透明、口径不一、难以横向比价	后台可配置的「品类→维度→档位」结构化计价体系，投标时自动算价并留存明细快照
需求 Demo 生成	软件类需求「纸上谈兵」，客户看不到成品长什么样	三段式 AI 流水线自动生成可交互的 HTML/CSS/JS 演示原型，支持反馈迭代与版本管理

三个模块均运行在统一的技术栈上：后端为 Express + Drizzle ORM + PostgreSQL（[artifacts/api-server](#)），前端为 React + Vite（[artifacts/jiedanba](#)）；大模型调用统一封装在 [src/lib/llm.ts](#)，支持后台配置多个 OpenAI 兼容供应商（如 DeepSeek），按激活状态排序自动故障转移（failover）。

术语速查：LLM=大语言模型；SSE（Server-Sent Events）=服务器向浏览器单向推送的流式协议，用于「打字机式」逐字输出；ReAct=让模型「思考→调工具→看结果→再思考」的循环模式；OPC=平台上的接单服务者；发单方=发布需求的客户。

第一章 需求分析智能体

1.1 功能定位与业务价值

需求分析智能体是一个嵌入需求发布/编辑页面的对话式 AI 助手。它像一位资深需求分析师，通过多轮引导式提问，帮客户把一句模糊的想法（「我想给员工做个 AI 培训」）梳理成结构完整的需求文档，并在结束时**一键回填发布表单**——标题、类型、预算区间、技能标签、抢单截止日、交付日、里程碑拆分全部自动填好。

业务价值：① 降低发单门槛，客户不需要懂「需求文档」怎么写；② 提升需求质量，OPC 拿到即可动手，减少来回澄清；③ 时间/预算等关键字段由工具计算而非模型「拍脑袋」，保证数据可靠。

1.2 整体流程

1. 用户在发布页打开对话面板，前端以 `sessionKey`（或 `demandId / conversationId`）发起 `POST /api/agent/demand-analysis/chat`。
2. 后端查找/创建会话（`agent_conversations`），按 `sceneKey` 从 `agent_configs` 加载场景化 system prompt，并动态注入：技能库文本（`skillRegistry`）、编辑模式下的当前需求数据、关联客户需求详情、自动收尾提示等上下文块。
3. 进入 **ReAct 工具调用循环**（最多 10 轮）：模型每轮可发起工具调用（查时间、拉模板、校验工期、估预算……），后端执行工具、把结果作为 tool 消息回传，直到模型产出最终文本回复。
4. 最终回复通过 **SSE 流式** 逐 token 推给前端；若回复中包含 `form_suggestion_json:` 标记，后端先用 `accumulated` 权威数据修补其中的日期/里程碑字段再下发。
5. 前端解析出表单建议并渲染「一键填入表单」卡片，用户确认后回填发布表单；全部消息（含工具调用与结果）持久化到会话记录，跨轮、跨页面刷新可复用。

1.3 核心机制说明

1.3.1 场景化 system prompt (`agent_configs`)

每个使用场景在 `agent_configs` 表中有一条配置（按 `sceneKey` 区分），包含系统提示词、启用开关等；提示词支持版本管理（`agent_config_prompt_versions`）。同一套对话引擎因此可复用于多个场景：

sceneKey	用途	特殊约束
demand_analysis	发单方新建需求的默认分析场景	全量工具
v2_demand_analysis	V2 客户需求分析	工具白名单（5 个，见 1.6）
v2_admin_opc_demand	运营基于客户需求创建 OPC 需求	仅管理员；白名单 7 个工具
v2_outsource_split / v2_admin_opc_milestone	需求拆分 / 里程碑助手	仅管理员；无工具（纯对话）

1.3.2 ReAct 工具调用循环（最多 10 轮）

后端在 MAX_TOOL_ITERATIONS = 10 的循环中反复调用 callLLM(messages, tools)：若返回 toolCalls，则逐个执行工具（executeTool），把结果以 tool 消息追加进上下文再进入下一轮；若返回纯文本，则切换为 streamLLM 流式输出最终回复。为防止历史损坏，持久化时保证 assistant(tool_calls) 与 tool 结果成对原子写入，读取时用 sanitizeHistory 清理孤儿工具调用（否则会导致 LLM 供应商返回 400）。

1.3.3 工具集清单

工具名	作用	要点
<code>get_current_time</code>	获取服务器当前北京时间	所有日期推算以此为基准，避免模型使用训练截止日期
<code>get_demand_types</code>	获取需求分类列表	优先返回数据库 <code>cat_categories</code> 活跃分类，兜底静态五类
<code>get_requirement_template</code>	按类型获取需求文档模板	优先用分类表的 <code>doc_template</code> ，否则用内置五套分模块引导模板
<code>get_skill_tags</code>	获取技能标签库	优先数据库 <code>cat_tags</code> ，兜底 40 个静态标签
<code>get_opc_levels</code>	OPC 等级说明与预算上限	C≤¥3,000 / B≤¥20,000 / A 与无限 ≤¥200,000
<code>search_opc_candidates</code>	按等级/关键词搜索 OPC（邀请模式）	查库返回≤20 人，指示模型以多选题呈现给运营
<code>validate_timeline</code>	校验交付日期合理性，倒推抢单截止日	可解析「3个月/2周/10天」等中文相对时间；按复杂度要求最少 7/14/21 天
<code>suggest_milestones</code>	生成里程碑拆分建议	需在 <code>validate_timeline</code> 通过后调用，复用其返回日期
<code>estimate_budget</code>	按类型/复杂度/工期估算预算区间	供第四阶段给用户预算参考
<code>perform_self_check</code>	自检轮次计数器	服务端统计历史调用次数返回 {round, max_rounds=10}，防自检死循环
<code>get_linked_demand_details</code>	拉取关联客户需求全文	服务端直接查库执行；实际上系统已把关联需求预注入 system prompt 做双保险

1.3.4 accumulated 权威数据机制（防日期幻觉）

LLM 在生成 JSON 时可能「顺手编」日期。为此后端维护一个 `accumulated` 累加器：`validate_timeline`（且 `isReasonable=true`）与 `suggest_milestones` 的工具返回值被视为 `bidDeadline / deadline / milestones` 三个字段的权威来源。在把 `form_suggestion_json` 下发给前端前，`injectAccumulatedData()` 会解析 JSON 并注入累加器的值：`bidDeadline`（抢单截止日）**无条件覆盖**模型写的值；`deadline` 与 `milestones` 则在模型未填写（或里程碑为空数组）时补齐——即模型自行填写的交付日/里程碑不会被强制替换。累加器还会在每次请求开始时**从历史消息中回放预填充**——即使用户在新一轮才说「给我表单」、模型不再重新调工具，上一轮的工具结果依然生效（跨轮复用）。

1.3.5 自动收尾（wrap-up）注入

若对话已达 8 个用户轮次仍未产出 `form_suggestion_json`，或用户消息命中「写文档 / 差不多了 / 帮我填」等收尾关键词，后端向 system prompt 追加强制指令，要求模型本轮必须输出表单 JSON，避免无限追问。

1.4 前后端交互方式

1.4.1 接口清单

接口	说明
<code>POST /api/agent/demand-analysis/chat</code>	核心对话接口，SSE 流式响应；请求体含 message、sessionKey/conversationId/demandId、sceneKey、mode(new/edit)、existingDemandData、linkedClientDemandId、agentContext
<code>GET /api/agent/demand-analysis/status</code>	查询智能体是否启用（控制入口按钮显隐）
<code>GET /api/agent/demand-analysis/history/:demandId</code> <code>GET .../history/session?sessionKey=</code>	按需求或会话键加载历史消息，面板打开时恢复上下文

1.4.2 SSE 事件协议

事件 type	含义
<code>conversation_id</code>	会话建立后立即下发，前端保存用于后续请求
<code>tool_call</code>	模型正在调用某工具，前端插入「正在查询…」指示气泡
<code>token</code>	最终回复的增量文本，前端逐字渲染
<code>done</code> / <code>error</code>	本轮结束 / 出错（错误也会持久化为一条助手消息）

1.4.3 结构化输出与表单回填

前端 `AgentChatPanel.tsx` 在流式渲染时实时解析三种内嵌标记（用括号配对算法容错提取 JSON）：

标记	用途
<code>form_suggestion_json:</code>	表单建议（标题/类型/描述/预算区间/技能标签/OPC等级/两个截止日/里程碑/发布模式/邀请 OPC），渲染为「一键填入表单」卡片，经 <code>onFillForm</code> 回填
<code>doc_update_json:</code>	编辑模式输出，仅更新需求描述字段，经 <code>onDocUpdate</code> 应用
<code>option_choices_json:</code>	选择题（单选/多选按钮组），降低用户打字成本

此外前端会做类型归一化（新分类码 CG/SA/TK/BO/OTHER → 旧表单五类），并剥离 DSML 等模型内部结构化标记，保证用户只看到干净文本。

1.5 数据存储结构

表	关键字段	说明
agent_configs	sceneKey、systemPrompt、isEnabled	每场景一条配置；未启用返回 503
agent_config_prompt_versions	configId、promptText、版本信息	提示词历史版本
agent_conversations	userId、demandId、sessionKey、linkedClientDemandId、messages(JSONB)	整段对话（user/assistant/tool 消息、toolCalls、reasoningContent、时间戳）以 JSON 数组存储；三键（conversationId→demandId→sessionKey）优先级查找复用
llm_providers	baseUrl、apiKey、defaultModel、isActive	LLM 供应商配置，激活者优先、失败自动切换，环境变量 DeepSeek 兜底

1.6 权限与触发条件

控制点	规则
登录鉴权	所有接口 requireAuth ；消息为空返回 400
管理员专属场景	v2_outsource_split 、 v2_admin_opc_demand 、 v2_admin_opc_milestone 仅 admin 角色可用（403）
启用开关	场景配置不存在或 isEnabled=false 时返回 503，前端隐藏入口
V2 场景工具白名单	v2_demand_analysis 仅允许 5 个工具（时间/分类/模板/预算/自检）；v2_admin_opc_demand 额外放开关联需求详情与 OPC 等级；拆分/里程碑场景无工具

第二章 报价卡

2.1 功能定位与业务价值

报价卡把「一口价」变成**结构化、可解释、可比对**的报价：平台后台按业务品类预先配置计价维度和档位，OPC 投标时像「点菜」一样勾选档位，系统自动算出最终价并留存完整计算明细；发单方端则以分层账单和多卡对比表呈现。

业务价值：① 报价口径统一，杜绝随意喊价；② 每一分钱可解释（基准项+调整系数+维护包），建立信任；③ 发单方可在同一维度体系下横向比较多个 OPC 的报价，决策效率高；④ 运营也可直接对客户需求出报价卡，支撑「平台报价」模式。

2.2 整体流程

- 后台配置**：管理员在「报价卡配置」中按品类（关联 `cat_categories` 赛道，兼容旧五类）维护三层维度（基准层 base / 调整层 adjustment / 可选层 optional），每个维度下配置若干档位（tier），档位携带 `basePrice`（基准层，元）或 `coefficient`（调整层，乘法系数）。
- OPC 投标算价**：OPC 在竞标页按需求品类加载配置，逐维度选择档位；前端实时计算最终价（见 2.3），提交投标时把 `quotedPrice`、逐项 `breakdown` 明细和完整 `quoteCardSnapshot` 计算快照一并写入 bids 记录。
- 运营直接报价**：管理员针对客户需求（或招标单）通过 `POST /api/v2/quotation-cards` 直接创建报价卡（总价+明细+备注），需求处于「报价中」状态时自动通知发单方。
- 发单方查看与比选**：发单方在需求详情中查看分层账单（`BreakdownDisplay`），或在多个投标间用对比表（`QuoteCardCompareView`）逐维度横向比较，选择中意的报价。

2.3 核心机制说明

2.3.1 品类 → 维度 → 档位配置体系

配置数据存于两张表：`quote_dimensions`（维度：category/catCategoryId、layer、code、label、sortOrder、isActive）与 `quote_tiers`（档位：dimensionId、tier、tierLabel、basePrice、coefficient、description）。后台读取接口 `GET /api/admin/quote-card/config` 将维度按品类分组、按层归桶（base/adjustment/optional），品类分组优先使用新赛道体系 `catCategoryId`，并向旧品类字符串（content/software/...）双向映射兼容。

2.3.2 四层计价模型（OPC 投标端 finalPrice 计算）

```

rawBase      =  $\Sigma$  所选基准层档位.basePrice           // 基准层合计
calibratedBase = round(rawBase  $\times$  (1 + 自调百分比/100)) // OPC 自调, 钳制在  $\pm 20\%$ 
factorProduct =  $\Pi$  所选调整层档位.coefficient          // 系数连乘
adjustedPrice  = round(calibratedBase  $\times$  factorProduct)
maintenanceFee = round(adjustedPrice  $\times$  维护包费率)      // 可选层
finalPrice     = adjustedPrice + maintenanceFee

```

提交时序列化两份数据：**breakdown**（人类可读的逐项账单：基准项金额、「综合调整（ $\pm N\%$ ）」、「调整系数（ $\times 1.20$ ）」影响额、「维护包（档位）」、备注）与 **quoteCardSnapshot**（机器可读快照：

baseLayers/adjustLayers 各档位选择、adjustmentPercent 及理由、

rawBase/calibratedBase/factorProduct/adjustedPrice/maintenanceFee/finalPrice 全链路中间值）。快照保证日后配置变更也不影响历史报价的可追溯性。

2.3.3 发单方展示与对比

BreakdownDisplay 按明细项的命名约定自动分组渲染：金额非 0 且非特殊项 $\rightarrow$ 基准层卡片；「综合调整（ $\cdots$ ）」 $\rightarrow$ 微调卡片；金额为 0 的维度项 + 「调整系数（ $\times \cdots$ ）」 $\rightarrow$ 调整层卡片（并回查配置显示每项系数）；「维护包（ $\cdots$ ）」 $\rightarrow$ 可选层卡片；最后合计与备注。**QuoteCardCompareView** 汇集所有含快照的待审投标，取各家快照的维度并集生成对比矩阵行，每列展示该 OPC 的档位选择/系数/小计与最终报价；价格展示按佣金率还原（ $\text{价格} \div (1 - \text{commissionRate})$ ，默认 10%）；自动标注「☒ 最低价」「超出预算」「低于下限」，未填报价卡的纯方案申请单独列出。

2.4 前后端交互方式

接口	权限	说明
GET /api/admin/quote-card/config[?category=]	admin	读取全部/单品类维度档位配置
POST /api/admin/quote-card/dimensions PUT/DELETE /api/admin/quote-card/dimensions/:id	admin	维度增删改（Zod 校验）
POST /api/admin/quote-card/tiers PUT/DELETE /api/admin/quote-card/tiers/:id	admin	档位增删改
GET /api/v2/quotation-cards? clientDemandId= tenderId=	登录用户	按父对象查报价卡列表（联查创建人昵称）
POST /api/v2/quotation-cards	admin	创建报价卡： parentType(client_demand/tender)+totalPrice+breakdown+note
GET /api/v2/quotation-cards/:id	登录用户	报价卡详情
PATCH /api/v2/quotation-cards/:id	admin	更新总价/明细/备注

2.5 数据存储结构

表	关键字段	说明
quote_dimensions	category、catCategoryId、layer(base/adjustment/optional)、code、label、sortOrder、isActive	计价维度；新赛道 ID 与旧品类字符串双轨兼容
quote_tiers	dimensionId、tier、tierLabel、basePrice、coefficient、description	档位：基准层用 basePrice，调整层用 coefficient
v2_quotation_cards	parentType、clientDemandId/tenderId、totalPrice、breakdown(JSONB)、note、createdBy	运营创建的报价卡；breakdown 为 {item, amount, note}[] 数组
bids.quote_card_snapshot	JSONB 快照（QuoteCardSnapshot 类型）	OPC 投标时的完整计价快照，供对比视图使用

2.6 权限与通知机制

场景	行为
配置管理	仅管理员（requireAdmin）可增删改维度与档位
报价卡创建/更新	仅管理员（requireAdmin）
报价卡查看	当前实现仅要求登录（requireAuth）， 未做 发单方/关联方归属校验，且列表接口不带过滤参数时会返回全部报价卡。这是已知的访问控制薄弱点，建议后续为读取接口增加父对象归属/管理员授权并强制要求 scoped 查询参数
创建时通知	若需求处于 quoting （报价中）状态，创建报价卡后刷新需求 updatedAt 触发红点，并向发单方发送 v2_quote_initiated 站内通知「运营方已发起报价…」
更新时通知	PATCH 后同样刷新红点并通知「运营方已更新报价…」

第三章 需求 Demo 生成

3.1 功能定位与业务价值

对软件开发类需求，平台在客户提交需求后**自动生成一个可点击、可交互的前端演示原型**（纯 HTML + CSS + JS + Tailwind CDN，含真实示例数据与至少两个可切换视图），让客户在签约前就「看到成品的样子」，并可用自然语言反馈驱动 Demo 迭代。

业务价值：① 把抽象需求可视化，大幅降低沟通成本与理解偏差；② 提升客户信任与成交率；③ 反馈闭环让需求在开发前就完成一轮验证；④ 版本留痕，管理员可回溯和打包下载任一版本。

3.2 整体流程（三种触发方式）

触发方式	入口	条件
① 提交需求自动触发	<code>POST /api/v2/client-demands/:id/submit</code>	需求类型为软件类（ <code>software / SA / sa</code> ）且需求详情 ≥ 200 字；满足则先插入 <code>generating</code> 状态占位行，再 <code>setImmediate</code> 后台异步生成
② 管理员手动重新生成	<code>POST /api/demands/:id/demo/regenerate</code>	仅 admin；无论是否已有 Demo 均可触发，状态置回 <code>generating</code>
③ 用户反馈驱动更新	<code>POST /api/demands/:id/demo/feedback</code>	发单方/管理员提交修改意见；先经 AI 分类器判定是否为有效的页面视觉/交互类意见（无效则友好提示驳回），有效则状态置 <code>updating</code> 并携带 <code>feedback</code> 增量重生成；生成中提交返回 409

单次生成的内部流程（generateDemo）

- 读取需求主记录与最新版本详情，拼装 demandInfo（标题/类型/预算/交付时间/详情）。
- 从 `agent_configs`（sceneKey= `demo_generation`）加载系统提示词（无配置则用内置默认版），并前置技能库上下文。
- 执行三段式 AI 流程（见 3.3），产出 `index.html / style.css / app.js` 三个文件。
- 将文件写入 `demo_projects`，状态置 `ready`，版本号 +1；若为反馈更新则追加 `revisionLog` 条目；版本文件同时归档到 `demo_project_versions`。

3.3 核心机制说明（三段式 AI 流程）

第一段：planDemo 方案规划

以「产品设计师」角色让 LLM 只输出文字版 UI 方案：产品定位、至少 2 个页面/版块清单、核心数据字段与真实感示例值、交互逻辑。规划失败不阻断，降级为无方案直接编码。

第二段：runCodingAgent 逐文件生成

以「资深前端工程师」角色进入最多 3 轮的工具调用循环，只提供两个工具：`write_file(filename, content)`（枚举限定 index.html / style.css / app.js 三个文件）与 `finish()`。反馈更新时会把当前版本文件全文和用户意见一并放入上下文做增量修改。每轮 finish 后执行**机械校验**：三文件齐全、app.js 通过 Node `vm.Script` 语法解析（只解析不执行）、HTML 含 body 标签与 Tailwind CDN；不通过则把错误清单回传给模型修复重写。模型不产生工具调用时会被文字提示「催促」。

第三段：safetyReviewJs 安全审查

静态正则检测 app.js 是否存在「DOM 获取后直接链式访问属性」的高危空指针模式（如 `getElementById('x').style`）；命中则再起一轮最多 2 轮的 LLM 审查，要求补齐 null 检查或可选链，修复稿必须再次通过 vm 语法校验才会替换原文件，否则保留原版——保证审查环节只可能变好不可能变坏。

技术约束设计：生成物刻意限定为无构建工具的纯三件套，且提示词明确要求 HTML 中**不写** style.css / app.js 的外链引用——因为前端预览时平台会把三个文件**内联**进 iframe 的 srcdoc，外链会 404。DOM 操作必须包在 DOMContentLoaded 里并做 null 检查，与第三段审查形成双保险。

3.4 前后端交互方式

接口	权限	说明
<code>GET /api/demands/:id/demo</code>	发单方本人 / admin	查询状态与文件；generating/updating 期间 files 返回 null，前端据此 轮询 直到 ready 后将文件内联渲染进 iframe 预览 (DemoPreviewModal)
<code>POST /api/demands/:id/demo/feedback</code>	发单方本人 / admin	提交修改意见 → AI 有效性分类 → 接受则后台重生成；返回 {accepted, message}
<code>POST /api/demands/:id/demo/regenerate</code>	admin	手动触发重新生成 (DemoManagePanel 管理面板)
<code>GET /api/demands/:id/demo/versions</code>	admin	版本列表：版本号、是否当前版、关联反馈、文件内容
<code>GET /api/demands/:id/demo/versions/:v/download</code>	admin	用 JSZip 将该版本文件打包为 <code>demo-v{N}.zip</code> 下载

3.5 数据存储结构与版本管理

表 / 字段	说明
demo_projects	每需求一条（demandId 唯一）：status（generating / updating / ready / error）、version（当前版本号）、files（JSONB：文件名→内容）、entryFile、dependencies、skillSnapshot（生成时的技能库快照）、revisionLog（JSONB 数组：{version, feedback, valid, timestamp}）、errorMsg
demo_project_versions	版本归档：demoProjectId（级联删除）、version、files、dependencies、createdAt；与 revisionLog 按版本号关联出「这一版是为哪条反馈生成的」

3.6 权限与触发条件

控制点	规则
OPC 隔离	OPC 角色访问任何 Demo 接口一律 403「OPC 无权访问 Demo」——Demo 属于客户与平台之间的售前资产
发单方归属校验	publisher 只能访问自己需求的 Demo（非本人 403，需求不存在 404）
管理员能力	重新生成、版本列表、zip 下载为 admin 专属
自动触发门槛	软件类 + 详情 ≥200 字；不满足时仅记录日志跳过，不打扰用户
并发保护	生成/更新中提交反馈返回 409；占位行插入用 onConflictDoNothing 防重
反馈质量闸门	classifyDemoFeedback 用 LLM 判定意见是否针对页面视觉/交互；分类器异常时默认放行（宁可多生成不可误拦截）